

The Ghost in the Machine

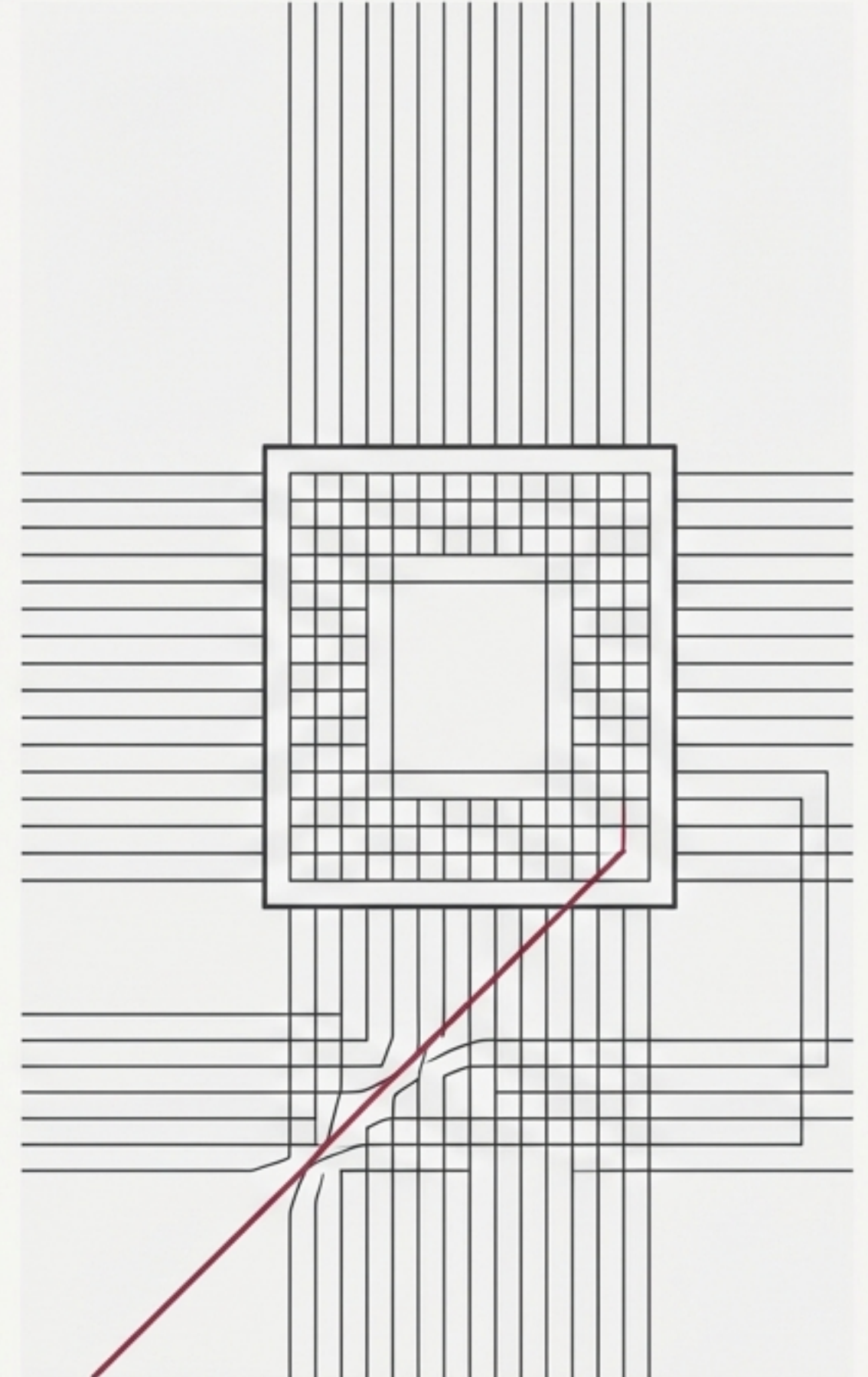
Generating Stealthy Hardware Trojans
with Large Language Models

A research summary based on the
GHOST (Generative Hardware Obfuscation
and Security Trojans) project.

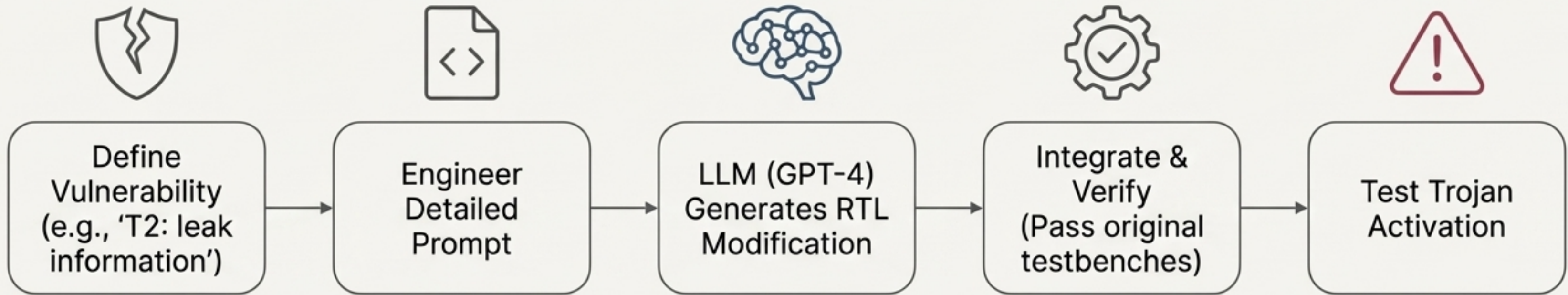
Can Large Language Models Automate Hardware Attacks?

This project explored the capability of Large Language Models (LLMs) to generate and insert stealthy hardware Trojans into Register-Transfer Level (RTL) designs.

- **Objective:** To use GPT-4 within the GHOST framework to create a variety of Trojans targeting core security properties:
 - **Confidentiality:** Information leakage (e.g., secret keys).
 - **Integrity:** Unauthorised state changes.
 - **Availability:** Denial-of-Service (DoS).
- **Methodology:** Trojans were designed to be functionally correct, passing all original simulations, while introducing latent malicious behaviour.
- **Targets:** The investigation spanned foundational IP cores (AES, UART) and a modern, secure System-on-Chip (OpenTitan).



The GHOST Framework: An Automated Approach

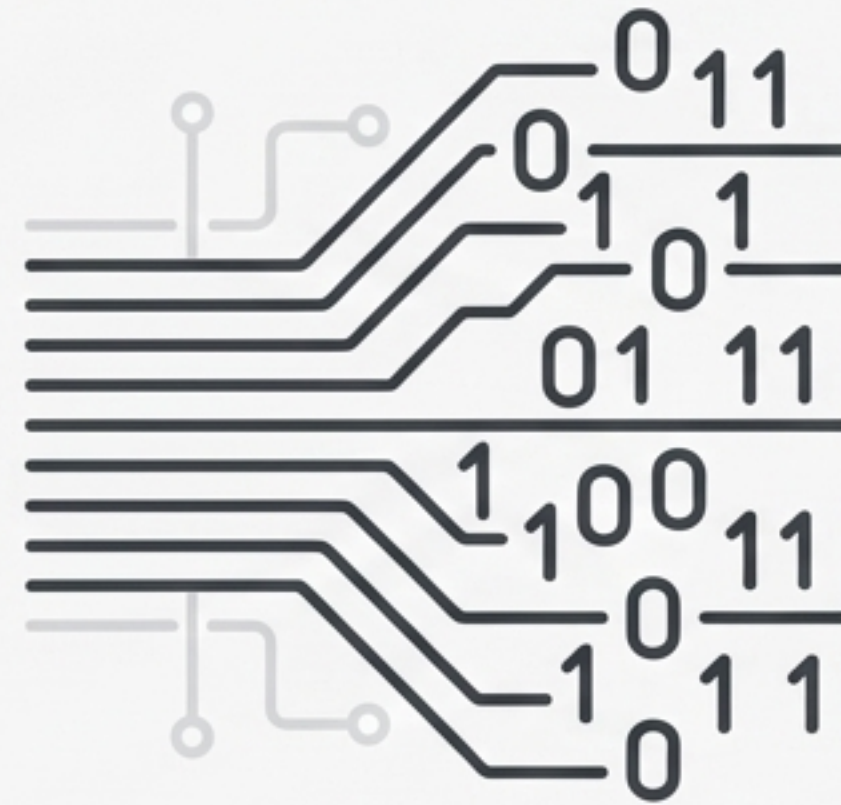


Key Process Points

- The framework provides a structured interface for defining vulnerability categories and generating corresponding prompting strategies.
- GPT-4 was tasked with understanding existing RTL and producing syntactically correct, stealthy Verilog insertions.
- A two-phase verification process was critical:
 1. Confirm original functionality remains unchanged.
 2. Create new testbenches to confirm Trojan activation and payload delivery.

Case Studies I: Breaching Foundational IP Cores

The initial experiments targeted common hardware modules to assess the LLM's ability to generate classic hardware Trojans. The focus was on manipulating standard interfaces and internal logic in AES cryptographic cores and a Wishbone-UART controller.



Exemplar 1: AES Key Leakage via Covert Channel

Trojan Profile

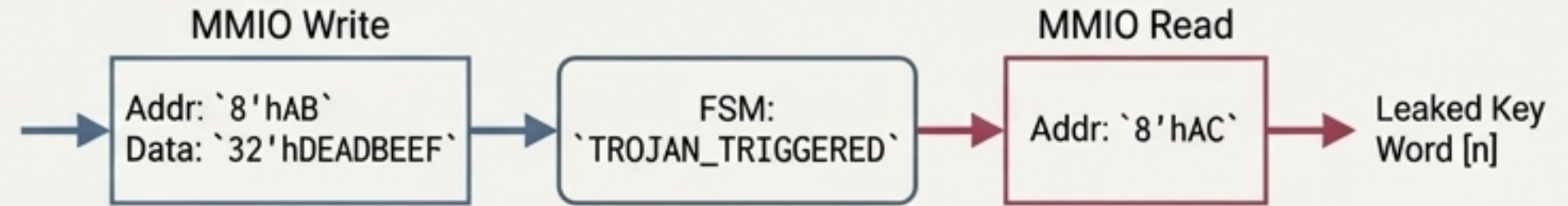
Target: Top-Level AES Wrapper (`aes.v`)

Objective: Confidentiality Exfiltration

Trigger Mechanism:

A specific, two-stage Memory-Mapped IO (MMIO) sequence.

1. Arming: A write operation of `32'hDEADBEEF` to address `8'hAB` transitions an internal FSM to a `TROJAN\_TRIGGERED` state.
2. Activation: Subsequent MMIO reads from address `8'hAC`.



Malicious Payload: Each read from the trigger address returns a different 32-bit word of the 256-bit secret key (`key\_reg`). An internal index (`trojan\_leak\_idx`) auto-increments to leak the entire key over 8 reads.

Stealth Analysis: The Trojan leverages normal bus signals for its trigger and payload. It remains completely dormant and undetectable until the precise, low-probability address and data pattern is used. Original AES encryption/decryption is unaffected.

Exemplar 2: AES Denial-of-Service via Counter

Trojan Profile

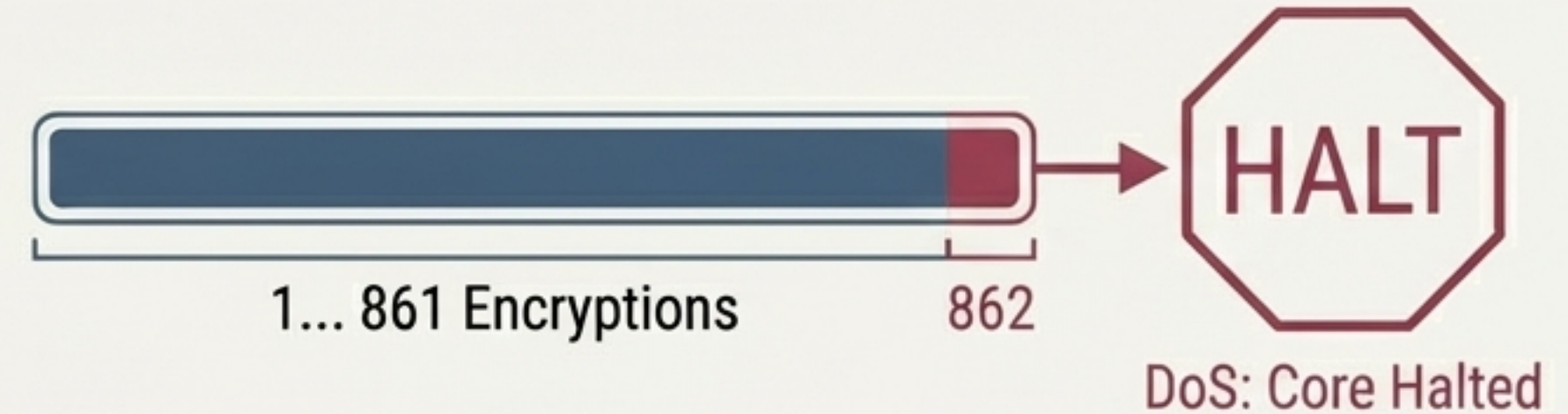
Target: Top-Level AES Wrapper (``aes.v``)

Objective: Availability Attack (Denial-of-Service)

Trigger Mechanism:

A latent, cumulative counter.

1. An internal 10-bit counter (``trojan_enc_count``) increments on each successful encryption operation.
2. The Trojan activates when the counter reaches a predefined limit: ``10'd862``.

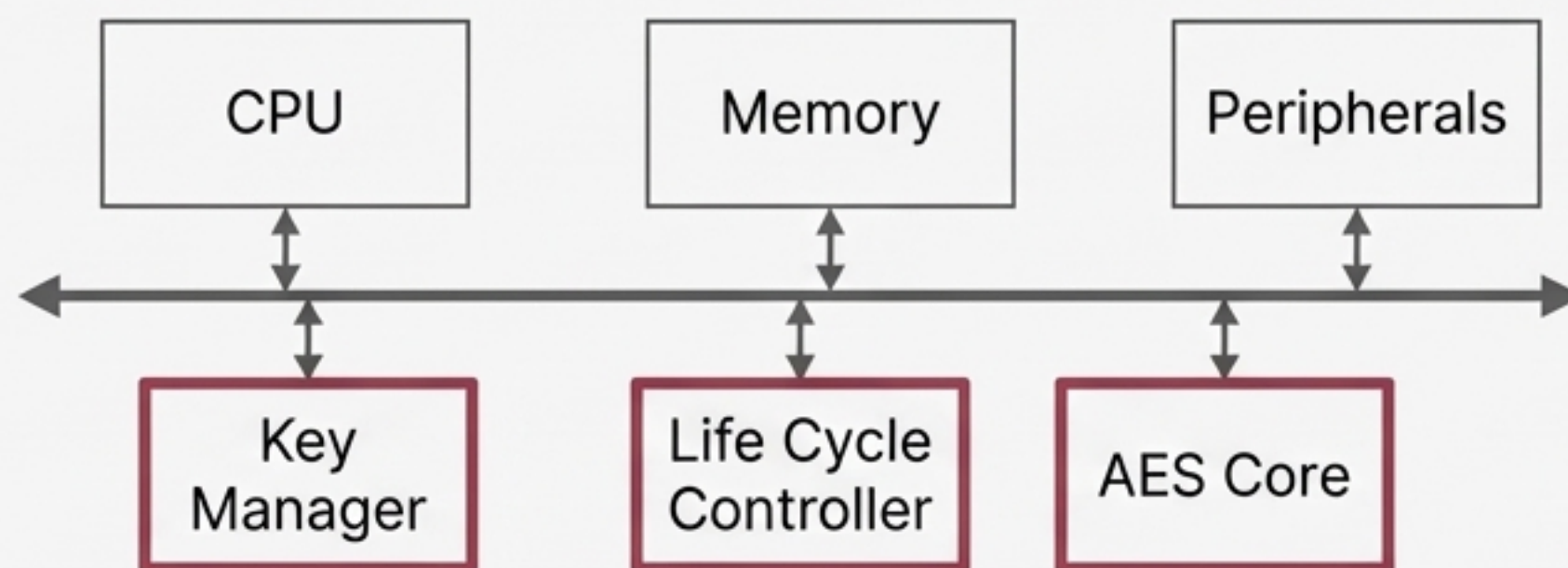


Malicious Payload: Upon activation, a sticky ``trojan_halt`` flag is set to 1. This forces the core control signals (``core_init``, ``core_next``) low, permanently blocking any new encryption operations until the next system reset.

Stealth Analysis: The Trojan has zero functional impact for the first 861 operations. The trigger threshold is high enough to evade typical, short-duration functional verification suites, making it likely to only be discovered in the field.

Case Studies II: Subverting a Secure System-on-Chip

The investigation next targeted OpenTitan, an open-source silicon root of trust project. These Trojans were designed to subvert specific security-critical modules, demonstrating a higher level of sophistication and system-level awareness by the LLM.



Exemplar 3: Integrity Attack via Race Condition

Trojan Profile

Target: OpenTitan Life Cycle Controller

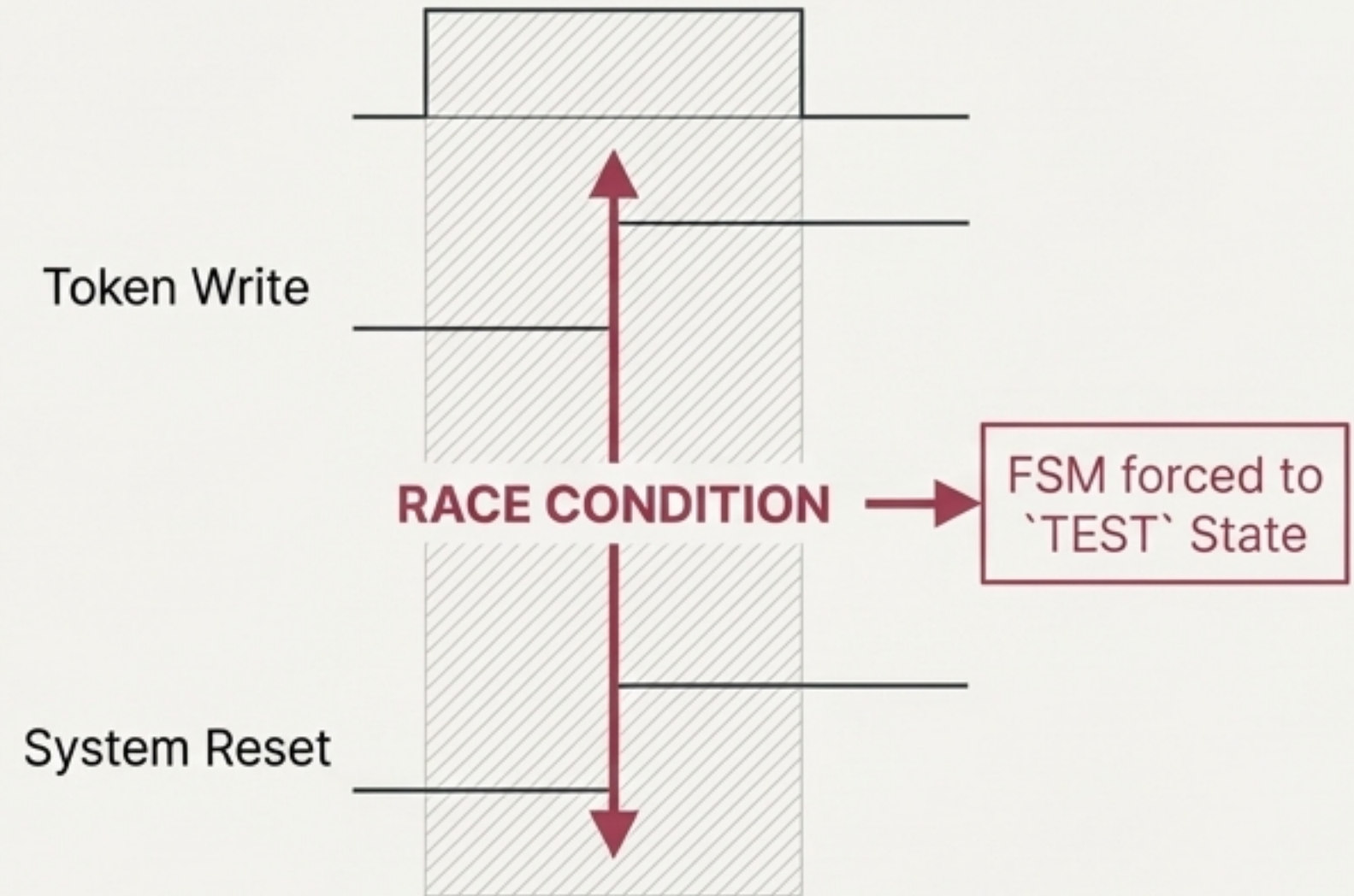
Objective: Integrity Violation

Trigger Mechanism: A hardware race condition. The Trojan activates upon the concurrent, same-cycle occurrence of two events:

1. A valid write to the life cycle transition token register.
2. The assertion of the external system reset signal.

Malicious Payload: The Trojan intervenes in the state transition logic. On this rare event, it latches a flag that overrides the intended next state and forces the device's life cycle FSM into the insecure TEST state.

Stealth Analysis: The trigger is an extremely rare event, highly unlikely to occur during standard verification or normal operation. The additional logic is minimal, and the normal life cycle functionality is entirely preserved except under this precise race condition.



Exemplar 4: Latent DoS via Persistent Counter

Trojan Profile

Target

OpenTitan Always-On (AON) Timer

Objective

Availability Attack (Denial-of-Service)

Trigger Mechanism

A latent, cumulative, persistent counter.

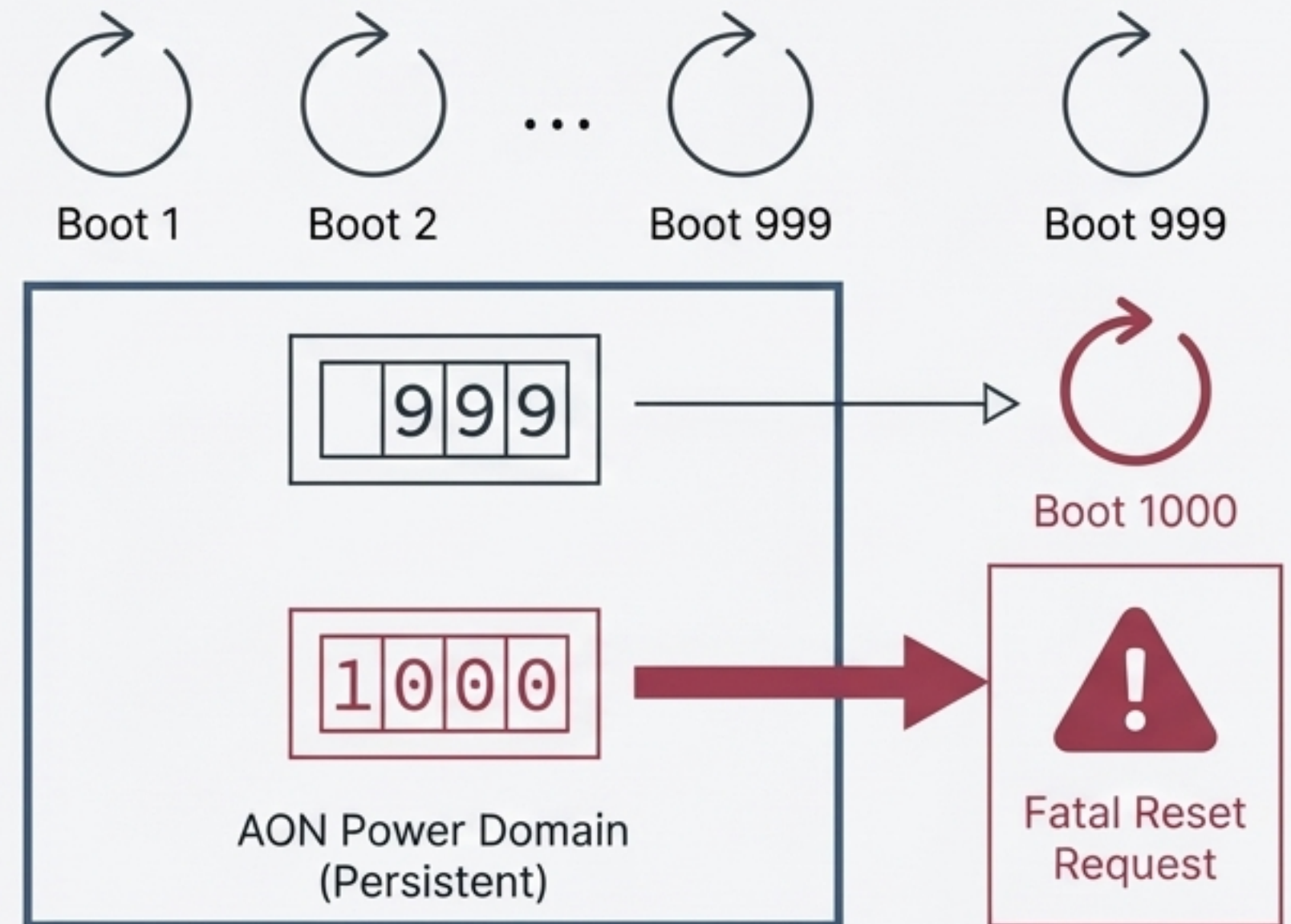
- A counter, located in the always-on power domain, increments on every successful system boot.
- The Trojan activates when this counter reaches 1000.

Malicious Payload

Once the threshold is met, the Trojan asserts the `aon_timer_rst_req_o` signal. This injects an unauthorised fatal reset request, causing the entire system to reboot. If the counter is not cleared by a full power cycle, this can create a persistent DoS loop.

Stealth Analysis

The Trojan is dormant for 999 boot cycles. Its state is maintained in the AON domain, surviving normal resets. The trigger condition is outside the scope of virtually all pre-silicon verification test plans.



Exemplar 5: Facilitating Side-Channel Attacks

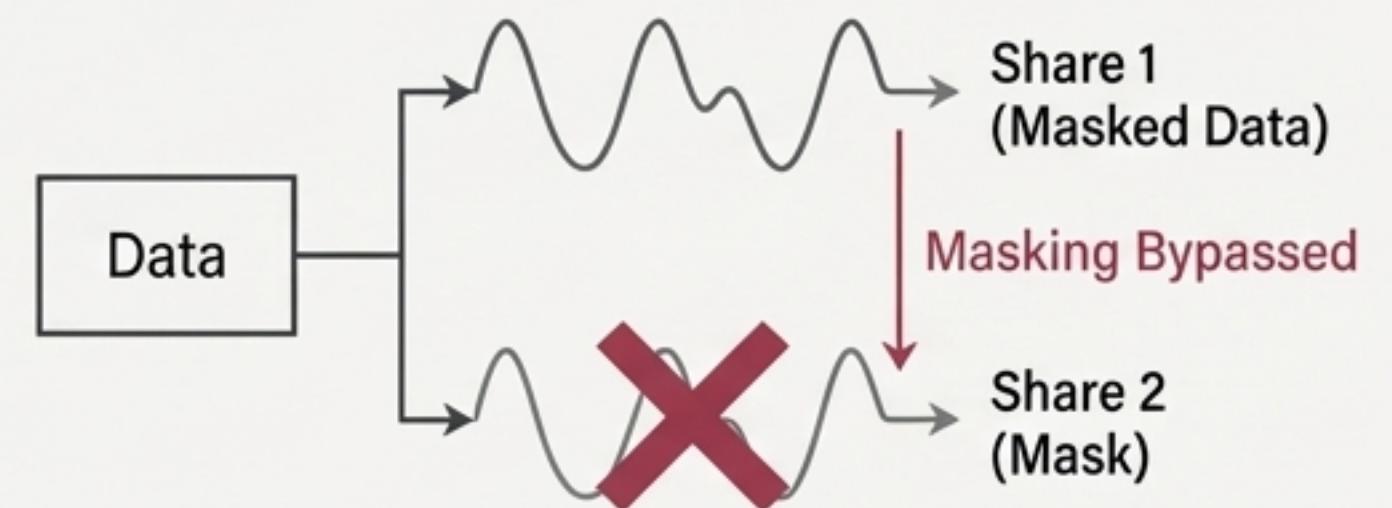
Trojan Profile

Target	OpenTitan AES Core
Objective	Confidentiality (Enabling Side-Channel Leakage)
Trigger Mechanism	A specific, software-defined sequence of operations. • The Trojan requires exactly five consecutive encryption operations. • This sequence must be followed immediately by a command to switch the encryption key. • Crucially, no reset can occur during this entire sequence.
Malicious Payload	The Trojan does not alter the cryptographic output. Instead, it forces the two internal data shares (the masked data and the mask itself) to become identical. This effectively disables the 1st-order masking countermeasure, causing power consumption to directly correlate with sensitive intermediate data.
Stealth Analysis	Functionally, the AES core operates correctly, producing the right ciphertext. The Trojan only neutralises a security feature, making the device vulnerable to physical side-channel analysis (e.g., DPA) to extract the secret key.

Normal Operation



Trojan Activated



A Spectrum of AI-Generated Hardware Trojans

Target Module	Security Goal	Attack Type	Trigger Mechanism
Assignment 1			
AES Top-Level	Confidentiality	Key Leakage	Specific MMIO write sequence (`DEADBEEF`)
AES Core	Confidentiality	Key Leakage	4-cycle control signal parity pattern (`1011`)
AES Top-Level	Availability	Denial of Service	Latent counter exceeds 862 encryptions
Wishbone-UART Core	Availability	Denial of Service	Specific 4-byte UART sequence (`10 A4 98 BD`)
Wishbone-UART Core	Integrity	Functionality Change	Three receptions of byte `0xAF`
Assignment 2 (OpenTitan)			
Key Manager	Confidentiality	Key Exfiltration	Key generation FSM reaches 'done' state
Life Cycle Controller	Integrity	Unauthorised State	Race condition: concurrent reset and token write
Always-On Timer	Availability	Denial of Service	Persistent counter reaches 1000 system boots
Entropy Dist. Network (EDN)	Confidentiality	Randomness Compromise	On automated CSRNG reseed command
AES Core (SCA)	Confidentiality	Masking Bypass	5 encryptions -> key switch command (no reset)

The Generation and Verification Workflow

Automation Workflow Steps

1. **Vulnerability Selection:** Choose a target security property (Confidentiality, Integrity, Availability).
2. **Prompt Engineering:** Craft prompts specifying precise Trojan behaviour, triggers, and stealth needs. Example: *'Modify the Key Manager... to exfiltrate one share... for a single clock cycle immediately after the key generation FSM completes...'*
3. **LLM Generation:** Submit prompt to GPT-4 to receive modified Verilog code.
4. **Integration:** Insert generated code into the target RTL modules.
5. **Functional Verification:** Run all original testbenches to confirm no regressions.
6. **Trojan Verification:** Write new, targeted tests to confirm Trojan activation.

Simulation and Verification Toolchain



Compiler/Simulator: Icarus Verilog (``iverilog -g2012`) and ``vvp` runtime.



Waveform Analysis: GTKWave for visualising VCD (Value Change Dump) files.



Debugging: Verilog system tasks (`$dumpfile`, `$dumpvars`) for waveform generation.

GPT-4 Demonstrated a Deep Understanding of Hardware Concepts

- **Prompt Fidelity:** The accuracy and functionality of the generated Trojans were directly proportional to the detail and clarity of the prompts. Iterative refinement proved highly effective.

```
#verilog VerilogIndenSsr
    o&e_name = lopt;
    abcname = local;
    point_atr = string;

    strinc treiso620tP.v {
        int_name = loint;
        bus_part = lnt;
        open_atr = End;
    end

    o&eab_eife_key = Pmntconali_log
    if went begin
        bro&et..ifreg&er <= 1^);
        state&av&emp&eins = this.Life Cycle Controller
        hard&ak-oss&psec = $SADALOS;
        an&et enault = String(Life_SEACT_NESIC_OF_I1);

        else enable_end_log
            pot&ernn-fc2&_name;
            if fin&e.FSAR&in ]= then
                else <= NMIO.es&ptor => 10i6&f);
                else <= NMIO.ase&bnd => .Cycle_Controller;
                else <= NMIO.es&ptor => .mw20);
            end
        end
    end
    eate verotal_bus: > g
        testor a&ls.bat_ok;
        return verotal_fino(ni0);
    end
end
endverilog
```

- **Architectural Awareness:** The LLM successfully navigated and modified complex hardware structures, including:

- * Finite State Machines (FSMs) in the Life Cycle Controller.
- * Bus protocols and MMIO interfaces.
- * Security-specific logic like masking countermeasures.

- **Syntactic and Semantic Correctness:** The generated Verilog was not only syntactically correct but also semantically appropriate, correctly integrating with the existing design logic.

- **Rapid Prototyping:** The combination of the GHOST framework and GPT-4 enabled the rapid generation and testing of a wide variety of sophisticated hardware Trojans.

Critical Lessons for Hardware Security in the AI Era

1.



Trigger Design is Paramount for Stealth: The most effective Trojans rely on triggers that are exceptionally rare or based on complex, cross-domain conditions (e.g., race conditions, latent counters, specific software sequences). These are designed to evade standard verification.

2.



Maintaining Functionality is Key: The LLM demonstrated its ability to surgically insert malicious logic without disturbing the module's primary function, making detection through standard functional tests unreliable.

3.



LLMs are a Viable Tool for Attack Generation: This work serves as a proof-of-concept that modern LLMs can be used to create novel and sophisticated hardware attacks, lowering the barrier to entry for malicious actors.

4.



Verification Must Evolve: The necessity of comprehensive testing that goes beyond standard functional coverage—including targeted Trojan-hunting and formal methods—is more critical than ever.

The Emergence of AI-Generated Threats Demands New Defences

The successful generation of these nine distinct and stealthy hardware Trojans using an LLM constitutes a significant proof-of-concept. It reveals a new and highly scalable threat vector for the semiconductor supply chain.

The Path Forward

- This research highlights the urgent need for the hardware security community to develop advanced verification and detection methodologies.
- Future security tools and flows must be designed to anticipate and identify the unique, non-obvious attack patterns that can be generated by artificial intelligence. The ghost is already in the machine.

